

Data source: <https://archive.ics.uci.edu/dataset/350/default+of+credit+card+clients>

Introduction and overview

This dataset consists of binary default probability of 30,000 customers in a month(Y) based on independent variables(X), 24 input features, such as gender, age and 6 months of credit history. This study aims to compare the training speeds and test set accuracies between MATLAB and Python in implementing the same 1 hidden layer neural network model with various hidden layer size and learning rates.

Overall, 77.9% (23,364) of the customers did not default, and the independent variables do not seem skewed with average hovering close to the median, apart from the left-skewed billed and paid amounts with averages generally 2-3 times of the medians, the billed amounts are also on average 10 times of the paid amount. To deal with this skewness and features with large values we performed z-score normalization.

Literature review

This dataset and the application of ML on credit data have been studied extensively, Yeh & Lien (2009) concluded Artificial Neural Networks ("ANNs") is the best performer among six data mining techniques by capturing non-linear relationships between variables but lack interpretability due to their "black box" nature. In contrast, Kvamme et al. (2018) implemented a convolutional neural network ("CNNs"), usually applied for "grid-like" data such as images, to predict default probability using temporal transaction data. However, this is not suitable for this dataset as it lacks temporal and spatial elements, demonstrating the need for tailored methodologies based on each dataset's characteristics. Further, our dataset may be fundamentally flawed, missing key independent variables for credit default, for instance, traditional credit factors like debt-to-income ratios (AE Khandani et al., 2010) and granular transaction data of the borrowers (Kvamme et al., 2018).

Methodology

This multilayer perceptron ("MLP") neural network consists of only one hidden layer, with dataset randomly split into training (80%) and testing (20%) sets, but no validation set because a 5-fold cross-validation is adopted (Ryan, 2025), which inherently evaluates model performance across different data subsets. Heaton (2017) laid out 3 basic rules for selecting hidden layer size, hence, we tested with the following: 12 is the midpoint between the input (24) and output (1); 17 comes from applying the second rule; 24 is the input size and the upper limit of the first rule; 32 is a value between the input size (24) and twice the input (48), following the third rule; and 48 is the largest value allowed by the third rule, staying under twice the input size.

To determine appropriate learning rates, Smith (2017) suggested the use of a simple "learning rate range test", which essentially involves increasing the learning rate gradually over a few epochs, until where accuracy starts to improve (minimum) and plateaus or drops (maximum). Therefore, learning rate starting from 0.001 is adopted, which gradually increases to 0.990, where we stopped seeing statistically significant improvements to the accuracies. Then we produced geometrically spaced rate between 0.001 and 0.99 for the grid search. Moreover, we have used Area under the curve (AUC) to evaluate our model as we have a binary classification problem together with an imbalanced dataset. We decide to use it over (EMP) (Verbraken et al., 2014), because EMP is dependent on class weight of the classification which we have changed to deal with the imbalanced data.

Comparison of MATLAB and Python

Overall, MATLAB outperforms Python in both accuracies and training time by 6% and 72% respectively on average. The significant reduction (68-76%) in training time is due to MATLAB being more optimised for executing loops. Note that the improvement in accuracies only average 1% when the learning rate of 0.001 is excluded, this is due to the low accuracies (0.444-0.548) in Python at this learning rate. We hypothesised this may be due to the use of float32 in Python, but the results remain unchanged when tested with float64. In Python, the average accuracy is 0.671 and increases to 0.72 when learning rate of 0.001 is excluded. Learning rate instead of hidden layer size correlate stronger with accuracies until it passes the learning rate of 0.18, showing it might be stuck in local maxima beyond this point. No statistically significant correlation between hidden layer size and accuracies is detected under a p-value test. The average training time in Python is 7.2 seconds, with significant increase as hidden layer size expands, increases by nearly 3 times as hidden layer size expands by 4 times from 12 to 48 neurons. As opposed to accuracies, hidden layer size instead of learning rate correlates strongly with training time.

Test Accuracy						Training Time (s)					
Neurons:12	33%	0%	4%	1%	2%	Neurons:12	-69%	-68%	-75%	-76%	-75%
17	35%	5%	1%	2%	2%	17	-73%	-71%	-74%	-73%	-73%
24	10%	11%	2%	2%	4%	24	-74%	-74%	-74%	-73%	-73%
32	35%	-18%	3%	2%	3%	32	-71%	-71%	-72%	-71%	-71%
48	0%	6%	0%	0%	-3%	48	-70%	-69%	-70%	-69%	-70%
LR:	0.001	0.006	0.031	0.180	0.990	LR:	0.001	0.006	0.031	0.180	0.990

In MATLAB, the average accuracy is 0.7, higher than Python, but no significant improvement (0.729) by

excluding the learning rate of 0.001. Like Python, accuracies correlate stronger with learning rate than hidden layer size, which also falls after passing 0.18. In contrast to Python, there is no statistically significant increase in accuracies with bigger hidden layer. The average training time is 2 seconds, more than 70% faster than Python. For comparison, although training time in MATLAB also correlates with hidden layer size instead of learning rate, the increase in training time is only 2.6 times when hidden layer size expands by 4 times.

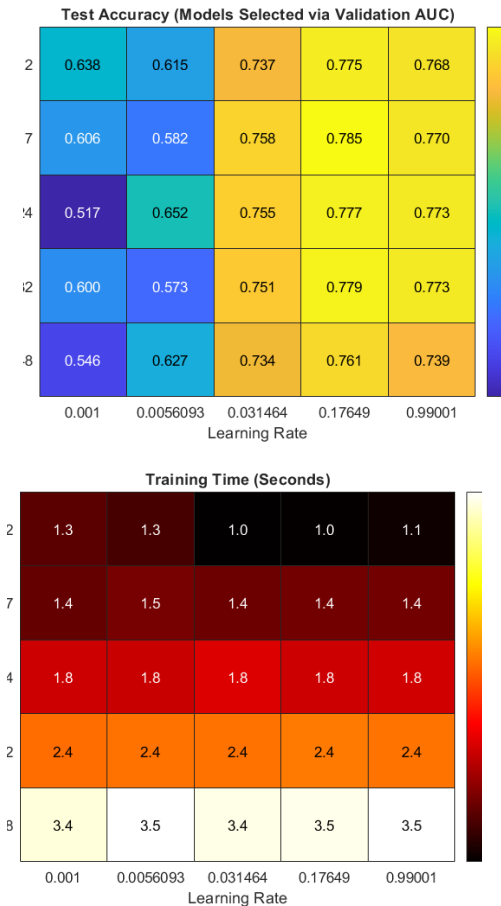


Figure 1. MATLAB Accuracy and training time

Conclusion and reflection

MATLAB clearly outperforms Python in both speed and accuracies, but this model's high false negative rate remains a critical issue, future work could address this through techniques such as class balancing (e.g. SMOTE), hyperparameter tuning or using weighted loss functions to penalise false negatives more heavily. The binary classification of default may have also inherently limited this model's accuracy, a probabilistic approach could offer a more nuanced risk assessments and fit into bank's risk models (Yeh & Lien, 2009). Finally, the non-parametric nature of ML models poses challenges for transparency, as noted by Moscatelli et al. (2020). While these models excel at capturing complex patterns, their lack of interpretability restrict their applicability in practice in credit risk modelling as regulatory authorities require complete transparency.

References:

1. I-C. Yeh and C-H. Lien. The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients. *Expert Systems with Applications*, 36(2, Part 1):2473-2480, 2009.
2. M. Moscatelli, F. Parlapiano, S. Narizzano, and G. Viggiano. Corporate default forecasting with machine learning. *Expert Systems with Applications*, 161:113567, 2020.
3. A.E. Khandani, A.J. Kim, and A.W. Lo. Consumer credit-risk models via machine-learning algorithms. *Journal of Banking & Finance*, 34(11):2767-2787, 2010.
4. H. Kvamme, N. Sellereite, K. Aas, and S. Sjursen. Predicting mortgage default using convolutional neural networks. *Expert Systems with Applications*, 102:207-217, 2018.
5. J. Heaton. Hidden Layers in Neural Networks. Heaton Research, 2017. Available at: <https://www.heatonresearch.com/2017/06/01/hidden-layers.html>
6. K. Ryan. (2025). "Neural Computing". City, University of London. 14 Feb. 2024. Lecture.
7. L. N. Smith, "Cyclical Learning Rates for Training Neural Networks," *2017 IEEE Winter Conference on Applications of Computer Vision (WACV)*, 464-472, 2017.
8. T. Verbraken, C. Bravo, R. Weber, and B. Baesens. Development and application of consumer credit scoring models using profit-based classification measures. *European Journal of Operational Research*, 238(2):505-513, 2014.

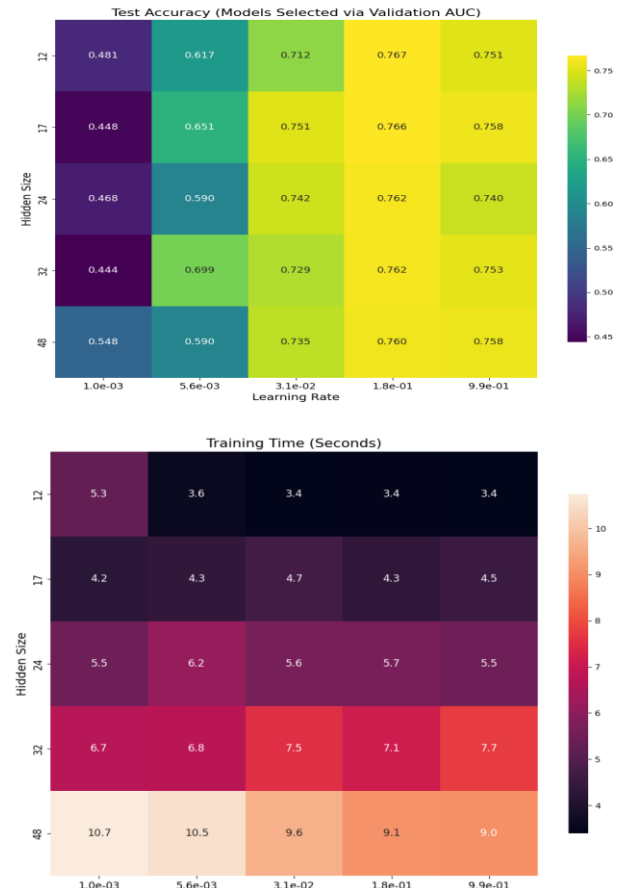


Figure 2. Python accuracy and training time